

TEAM D — FRONTEND

Smart Campus Lab & Resource Optimizer

Complete Frontend Structure, Role-Based Pages & Proposed Technology Stack

This document consolidates the Team D frontend plan: common pages, Student pages, Faculty pages, Lab Admin pages, role-based navigation, project folder structure, integration responsibilities and the proposed frontend technology stack.

Scope: Frontend/UI only. BFF implementation, backend implementation, database implementation, scheduling algorithms and routing algorithms are not Team D frontend responsibilities in this document.

1. Team D Frontend Responsibility

Main objective: Build the complete user-facing web application that makes the campus lab/resource system simple and usable for Students, Faculty and Lab Admins.

- Provide role-based dashboards and navigation.
- Allow Students and Faculty to search labs and create/manage their own bookings.
- Display booking status, queue information, allocated resources and route information.
- Provide operational monitoring and management screens for Lab Admin.
- Provide Admin-only resource and maintenance management.
- Provide Admin-only reports and utilization views.
- Build responsive, accessible and consistent UI components.
- Use mock data during Phase 1 and prepare the UI for later integration with the other teams.

Separation of responsibility: Team D presents information and user interactions. The authoritative data, scheduling decisions and routing calculations remain with their respective teams/services.

2. Role-Based Frontend Access

Area	Student	Faculty	Lab Admin
Login	✓	✓	✓
Dashboard	✓	✓	✓
Search Labs	✓	✓	✓ / operational view
Create own booking	✓	✓	Optional / as permitted
My Bookings	✓	✓	—
Booking Details	✓	✓	✓
Route / Campus Map	✓	✓	✓ / operational use
Live Monitoring	✓ limited status	✓ limited status	✓ full operational view
All Bookings	—	—	✓
Lab Management	—	—	✓
Resource Management	—	—	✓
Maintenance Management	—	—	✓
Reports	—	—	✓

Key rule: Student and Faculty interfaces should not expose Admin-only reports, resource management or maintenance controls.

3. Student Frontend Pages

Student navigation:

Dashboard → Search Labs → Lab Details → Create Booking → My Bookings → Booking Details → Route → Profile

Student Dashboard — /student/dashboard

- Available labs
- Upcoming bookings
- Current booking status
- Queue information
- Quick Book Lab action
- Recent notifications
- Route shortcut

Search Labs — /student/labs

- Date/time filters
- Capacity filter
- Equipment/capability filter
- Availability results
- View Details
- Book

Lab Details — /student/labs/:labId

- Lab name/location/capacity
- Availability
- Equipment/resources
- Lab status
- Book
- View route

Create Booking — /student/bookings/create

- Lab
- Date/time
- Required resources
- Purpose
- Review
- Confirm
- Validation and error states

My Bookings — /student/bookings

- Upcoming
- Pending/queued
- Active

- Completed
- Cancelled
- View details

Booking Details — /student/bookings/:bookingId

- Booking summary
- Status tracker
- Queue position
- Estimated start
- Allocated resources
- Route summary

Route — /student/route/:bookingId

- Source
- Destination
- Campus topology/map
- Highlighted route
- Distance
- Estimated travel time
- Route status

4. Faculty Frontend Pages

Faculty navigation:

Dashboard → Search Labs → Lab Details → Create Booking → My Bookings → Booking Details → Route → Profile

Faculty Dashboard — /faculty/dashboard

- Available labs
- Upcoming/active bookings
- Booking status
- Queue information
- Quick booking
- Notifications

Search Labs — /faculty/labs

- Date/time
- Capacity
- Equipment/capability
- Availability
- Lab result cards
- View Details

Lab Details — /faculty/labs/:labId

- Location
- Capacity
- Resources/equipment
- Availability
- Lab status
- Book
- Route

Create Booking — /faculty/bookings/create

- Lab
- Date/time
- Resources
- Purpose
- Review and confirm
- Validation
- Success/error states

My Bookings — /faculty/bookings

- Upcoming
- Active
- Queued

- Completed
- Cancelled
- View Details

Booking Details — `/faculty/bookings/:bookingId`

- Booking information
- Status
- Queue
- Estimated start
- Allocated resources
- Route

Route — `/faculty/route/:bookingId`

- Campus route
- Destination
- Distance
- Travel time
- Route status

Access restriction: Faculty should not see the Admin Reports, Resource Management or Maintenance Management navigation.

5. Lab Admin Frontend Pages

Admin navigation:

Dashboard → Labs → Resources → Maintenance → Bookings → Live Monitoring → Reports → Profile

Admin Dashboard — /admin/dashboard

- Total labs
- Available labs
- Occupied labs
- Labs under maintenance
- Active/pending bookings
- Queue size
- Resource availability
- Alerts
- Utilization summary

Lab Management — /admin/labs

- Lab list
- Lab ID/name
- Capacity
- Location
- Operational status
- View/update lab details
- View resources

Resource Management — /admin/resources

- All resources
- Resource ID/type
- Lab/location
- Operational status
- Maintenance status
- Availability
- Filters
- Add resource
- View details

Resource Details — /admin/resources/:resourceId

- Resource information
- Current status
- Lab
- Last updated
- Operational/maintenance action

- Confirmation feedback

Maintenance Management — /admin/maintenance

- Maintenance list
- Affected resources
- Affected labs
- Maintenance status
- Update status
- Maintenance information
- Operational/maintenance transitions

All Bookings — /admin/bookings

- All users' bookings
- Lab
- Date/time
- Status
- User
- Filters by lab/date/status
- Open booking details

Live Monitoring — /admin/monitoring

- Lab status
- Resource status
- Active bookings
- Queue
- Alerts
- Last updated
- Operational overview

Reports — /admin/reports

- Lab utilization
- Resource utilization
- Booking counts
- Completed/cancelled/rejected/queued counts
- Date range filter
- Lab/resource filters
- Charts and tables

6. Common / Shared Frontend Pages & Components

These are shared building blocks rather than separate copies for every role.

Shared area	Purpose
Authentication	Login, session state, logout, session-expired UI
Navigation	Role-aware sidebar/navbar and route protection
Notifications	Success, warning, error and informational messages
Loading states	Skeletons/spinners during data retrieval
Empty states	No bookings, no labs, no resources, no reports
Error states	Friendly error display and retry actions
Status badges	Available, occupied, maintenance, queued, confirmed, etc.
Booking components	Booking card, booking form, status tracker, queue status
Lab components	Lab card, lab details, availability indicator
Resource components	Resource card/table, resource status
Route components	Map/topology view, route summary
Charts/tables	Admin reports and operational data

Reuse principle: Student and Faculty can use the same BookingForm, BookingCard, BookingStatus and Route components while remaining separated by role-based routes and navigation.

7. Recommended Frontend Folder Structure

```
src/
  assets/
  components/
  common/
    Button/
    Modal/
    Table/
    StatusBadge/
    Loading/
    ErrorMessage/
    EmptyState/
  layout/
    Navbar/
    Sidebar/
  lab/
    booking/
    resource/
    monitoring/
    route/
    reports/
  layouts/
    AuthLayout/
    StudentLayout/
    FacultyLayout/
    AdminLayout/
  pages/
    auth/
      Login/
    student/
      Dashboard/
      Labs/
        LabSearch/
        LabDetails/
        Bookings/
        CreateBooking/
        MyBookings/
        BookingDetails/
      Route/
      Profile/
    faculty/
      Dashboard/
      Labs/
        LabSearch/
        LabDetails/
        Bookings/
        CreateBooking/
        MyBookings/
        BookingDetails/
      Route/
      Profile/
    admin/
      Dashboard/
      Labs/
        LabList/
        LabDetails/
      Resources/
        ResourceList/
        ResourceDetails/
        AddResource/
      Maintenance/
        MaintenanceList/
      Bookings/
        AllBookings/
        BookingDetails/
      Monitoring/
      Reports/
      Profile/
  routes/
```

```
■ ■■■ AppRoutes/  
■ ■■■ ProtectedRoute/  
■ ■■■ RoleRoute/  
■■■ services/  
■ ■■■ api/  
■ ■■■ mock/  
■■■ data/  
■ ■■■ users/  
■ ■■■ labs/  
■ ■■■ resources/  
■ ■■■ bookings/  
■ ■■■ routes/  
■■■ hooks/  
■■■ utils/  
■■■ types/  
■■■ App.jsx  
■■■ main.jsx
```

Why this structure? Pages are separated by user role, while reusable UI and domain components stay shared. This prevents unnecessary duplication and makes role-based access easy to maintain.

8. Route / Access Structure

Role	Example route	Access
Public	/login	Everyone before login
Student	/student/dashboard	Student only
Student	/student/labs	Student only
Student	/student/bookings	Student only
Faculty	/faculty/dashboard	Faculty only
Faculty	/faculty/labs	Faculty only
Faculty	/faculty/bookings	Faculty only
Admin	/admin/dashboard	Lab Admin only
Admin	/admin/resources	Lab Admin only
Admin	/admin/maintenance	Lab Admin only
Admin	/admin/bookings	Lab Admin only
Admin	/admin/monitoring	Lab Admin only
Admin	/admin/reports	Lab Admin only

Implementation rule: Use a ProtectedRoute for authentication and RoleRoute/role guards for Student, Faculty and Admin authorization.

9. Proposed Frontend Technology Stack

Note: The following is a recommended Team D frontend stack for Phase 1. It is a technical recommendation, not a claim that the uploaded project specification mandates these exact libraries.

Layer	Recommended technology	Purpose
Language	JavaScript / TypeScript	Frontend application development; TypeScript recommended for larger project
UI framework	React	Component-based frontend
Build tool	Vite	Fast development/build workflow
Routing	React Router	Role-based page routing and navigation
Styling	Tailwind CSS	Responsive and consistent UI styling
UI components	shadcn/ui or reusable custom components	Forms, dialogs, tables, cards and accessibility-friendly primitives
Charts	Recharts	Admin reports and utilization charts
Forms	React Hook Form	Booking/resource form handling
Validation	Zod	Frontend schema validation
Server data	TanStack Query	Fetching, caching and synchronization when real services are integrated
HTTP	fetch / Axios	Service communication when integration is added
Maps/topology	React Leaflet or a project-approved map/topology visualization	Map/topology visualization; use the project/team-approved option
Icons	Lucide React	Consistent UI icons
Testing	Vitest + React Testing Library	Unit/component tests
E2E	Playwright	End-to-end user-flow testing
Code quality	ESLint + Prettier	Linting and consistent formatting
Version control	Git + GitHub/GitLab	Team collaboration and versioning

10. What Team D Needs From Teams A, B & C

Source	Information Team D needs	Frontend usage
Team C	Users/roles	Login, role-aware UI
Team C	Labs/capacity/location/status	Search, Lab Details, Dashboard
Team C	Resources/availability/maintenance state	Lab Details, Admin Resources, Monitoring
Team C	Booking records/status/history	Create Booking, My Bookings, Booking Details
Team A	Queue position/requests ahead	Booking Details, Dashboard
Team A	Estimated start/scheduling state	Booking Details
Team A	Allocation/scheduling metrics	Booking Details, Admin dashboard/reports
Team B	Campus nodes/edges/topology	Campus Map / Route
Team B	Route/distance/travel time/status	Route page, Booking Details

Phase 1: Create mock data matching these structures. **Phase 2:** Replace mock data with the real integrated outputs/services.

11. Frontend Quality Requirements

Every important page should handle more than the happy path.

- Loading state — show skeleton/spinner while data is loading.
- Empty state — clearly explain when there are no labs, bookings or resources.
- Success state — show confirmation after successful actions.
- Validation state — show field-level and form-level validation.
- Error state — show understandable errors with retry where appropriate.
- Permission state — prevent unauthorized pages/actions from being displayed.
- Responsive design — desktop, laptop, tablet and mobile.
- Accessibility — labels, keyboard navigation, visible focus, readable contrast and clear messages.
- Consistent status indicators — Available, Occupied, Maintenance, Queued, Confirmed, Active, Completed, etc.

12. Final Team D User Flows

Student

Login → Student Dashboard → Search Lab → Lab Details → Create Booking → My Bookings → Booking Details → View Route

Faculty

Login → Faculty Dashboard → Search Lab → Lab Details → Create Booking → My Bookings → Booking Details → View Route

Lab Admin

Login → Admin Dashboard → Monitor Labs/Resources → Manage Resources → Manage Maintenance → Monitor All Bookings → Reports

Best integration demonstration

Login → Dashboard → Search Lab → Create Booking → Booking Details → Show booking status + queue + allocated resources + route → Admin later sees the same operational state through Monitoring/Reports.

13. Team D Development Checklist

- Set up React + Vite project and repository.
- Create common layout and reusable component system.
- Implement Login and protected routes.
- Implement Student pages.
- Implement Faculty pages.
- Implement Admin pages.
- Implement role-based navigation.
- Build mock data for users, labs, resources, bookings and routes.
- Build Booking Details as the main integration screen.
- Build Admin Resource Management and Maintenance pages.
- Build Admin Reports and Monitoring.
- Add loading, empty, success and error states.
- Make the UI responsive and accessible.
- Add frontend unit/component tests and important end-to-end flows.
- Freeze the data requirements/contracts needed from Teams A, B and C.
- Later replace mock services with integrated services/data.

Team D final outcome: A complete, role-based, responsive frontend that provides the user-facing experience for Students, Faculty and Lab Admin and is ready for later integration.